LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE CR002
PERTEMUAN 4

Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)

Disusun Oleh:

Arum Syafitri Puspita Pratiwi (20240801010)

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

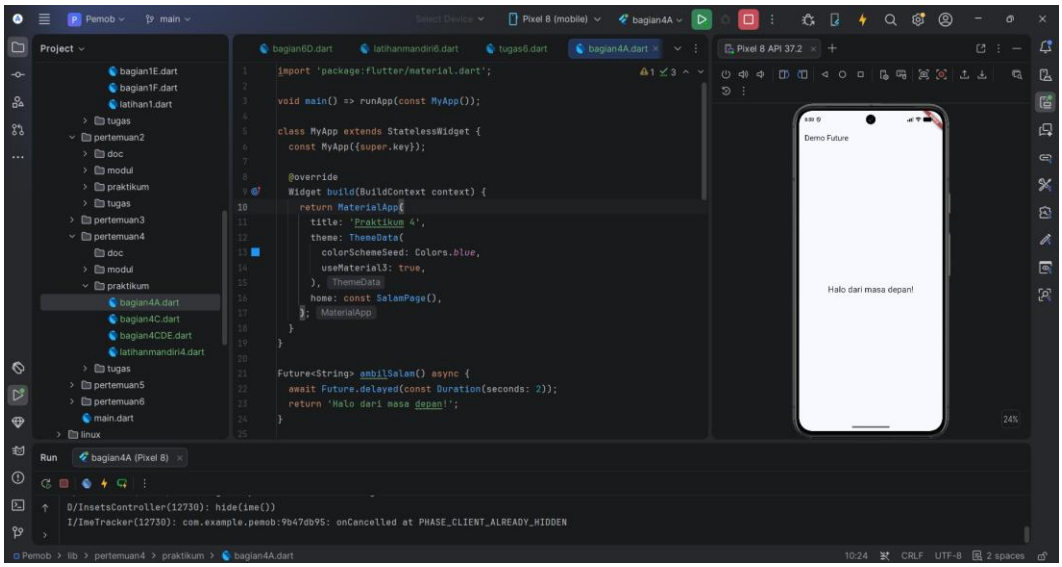
PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

Bagian A: Mengenal Future dan FutureBuilder (tanpa internet)



Praktikum bagian 4A ini bertujuan untuk memahami dasar pemrograman asinkron menggunakan Future dan menampilkan hasilnya secara dinamis pada antarmuka pengguna menggunakan widget FutureBuilder di Flutter. Program mensimulasikan proses penundaan waktu (delay) selama dua detik menggunakan Future.delayed untuk menampilkan teks salam setelah indikator pemuatan selesai.

Bagian B: Menyiapkan Paket HTTP

```
C:\Pemob>flutter pub add http
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
Changed 4 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

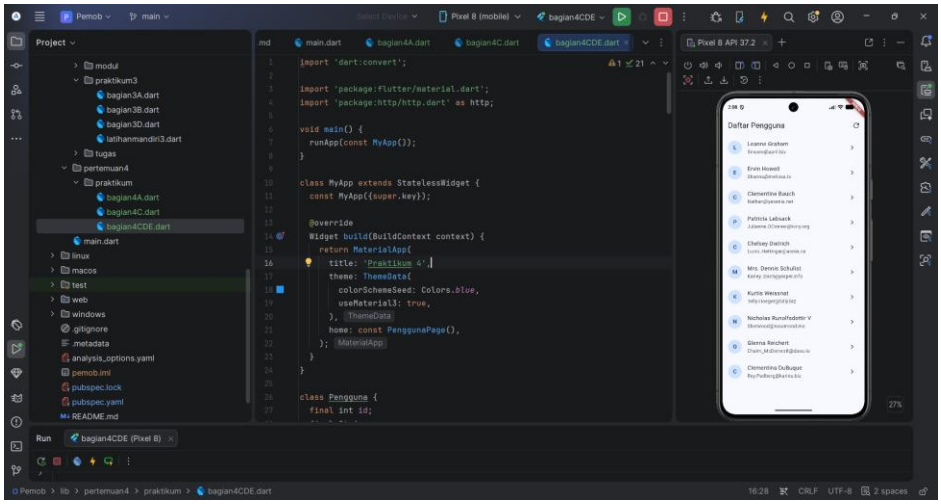
Praktikum bagian 4B ini berfokus pada persiapan proyek untuk melakukan komunikasi jaringan dengan menambahkan paket http ke dalam dependensi aplikasi. Selain itu, langkah ini juga mencakup pengaturan izin akses internet pada konfigurasi sistem operasi agar aplikasi dapat terhubung ke server luar atau REST API dengan lancar.

Bagian C: Model dan Fungsi Pengambil Data

Bagian D: Menampilkan Data dengan FutureBuilder

Bagian E: Halaman Detail

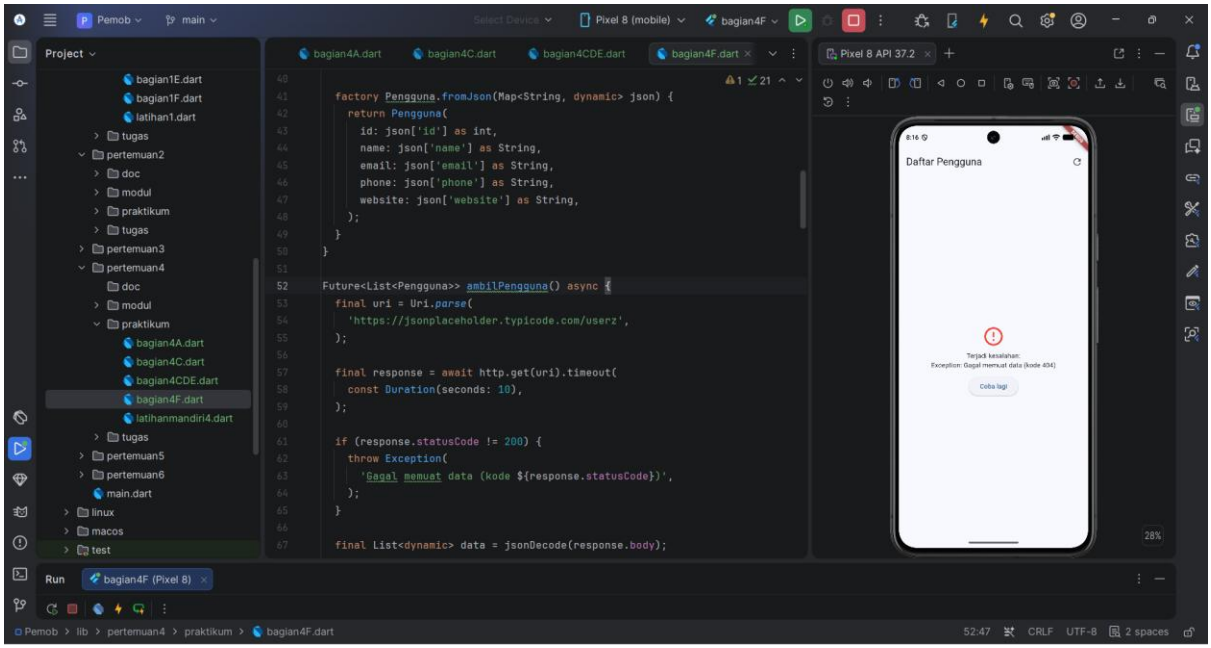
(digabung menjadi satu)



Praktikum bagian C, D, dan E ini melibatkan pembuatan class model Pengguna untuk memetakan data JSON dari REST API, serta fungsi ambilPengguna() untuk mengambil data secara asinkron. Data tersebut kemudian ditampilkan ke dalam antarmuka menggunakan FutureBuilder yang menangani berbagai kondisi seperti proses pemuatan, penanganan galat dengan tombol coba lagi, hingga menampilkan daftar pengguna dalam ListView dan halaman detail informasi lengkap saat salah satu pengguna dipilih.

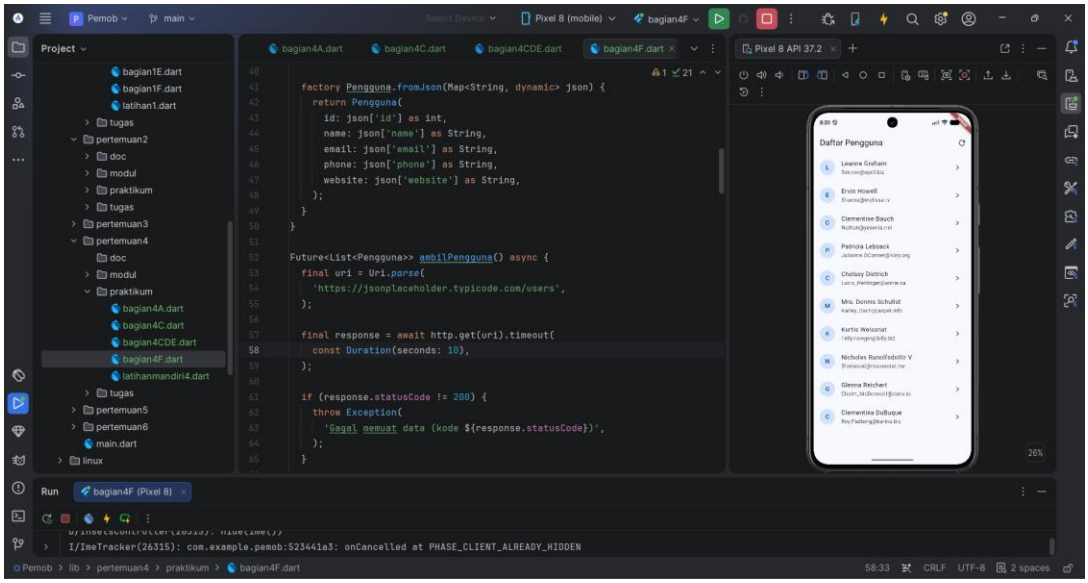
Bagian F: Percobaan Penanganan Galat

Percobaan yg pertama: Ubah URL menjadi /userz (halaman tidak ada). Amati pesan galat dengan kode 404, lalu kembalikan URL yang benar dan tekan Coba lagi.



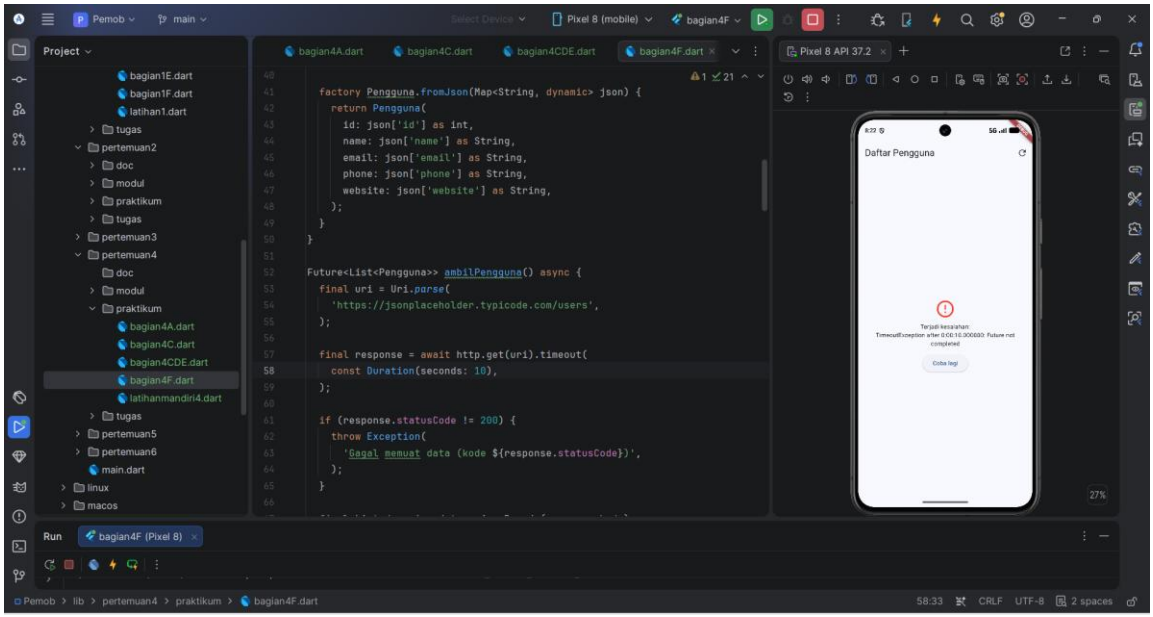
Praktikum bagian ini melakukan pengujian penanganan galat dengan mengubah URL endpoint menjadi /userz untuk mensimulasikan halaman yang tidak tersedia. Melalui pengujian tersebut, aplikasi berhasil mendeteksi kegagalan permintaan dan menampilkan pesan kesalahan berupa kode status 404 beserta tombol coba lagi pada antarmuka pengguna.

Setelah kembalikan URL yang benar dan tekan Coba lagi



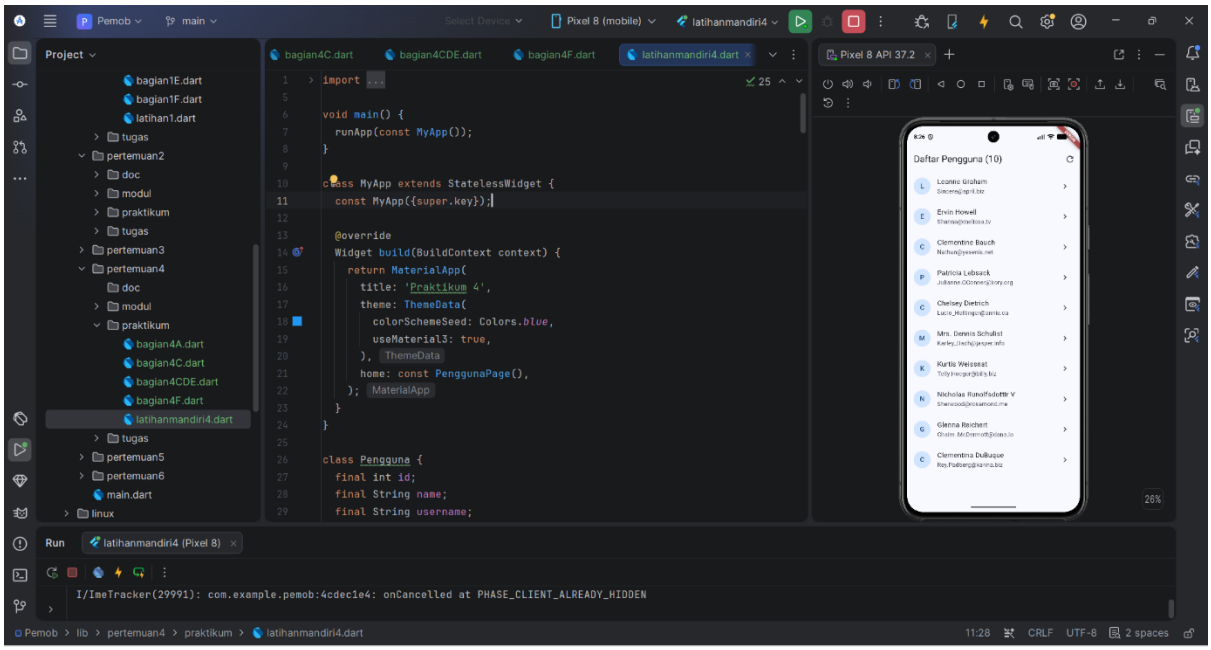
Setelah URL dikembalikan ke alamat yang benar dan tombol coba lagi atau muat ulang ditekan, aplikasi berhasil melakukan permintaan ulang (*re-fetch*) ke server REST API dengan sukses. Status koneksi kembali normal dengan kode respons 200, sehingga data daftar pengguna dari server berhasil dimuat dan ditampilkan kembali secara lengkap pada antarmuka aplikasi.

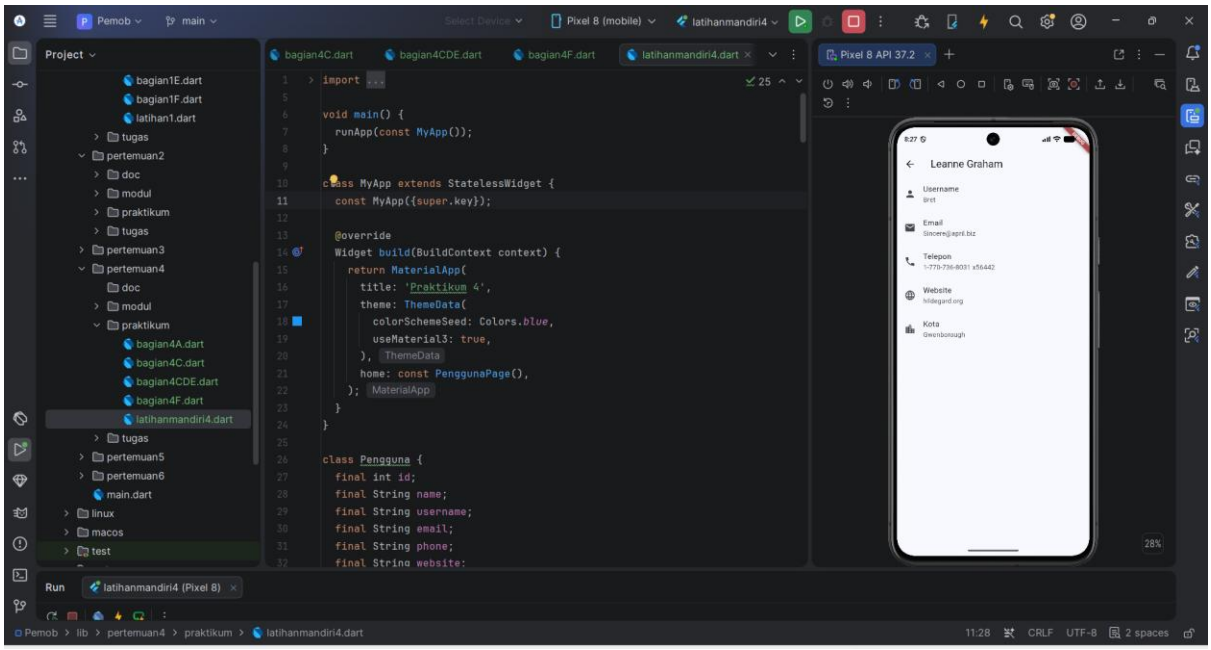
Cara kedua: Matikan koneksi internet perangkat/emulator lalu tekan tombol refresh. Amati pesan galat jaringan, nyalakan internet kembali, dan tekan Coba lagi



Praktikum bagian ini menguji penanganan galat saat perangkat kehilangan koneksi internet dengan mematikan jaringan lalu menekan tombol *refresh*. Melalui pengujian tersebut, aplikasi berhasil mendeteksi kegagalan koneksi dan menampilkan pesan galat jaringan (*TimeoutException*) beserta tombol coba lagi pada antarmuka pengguna.

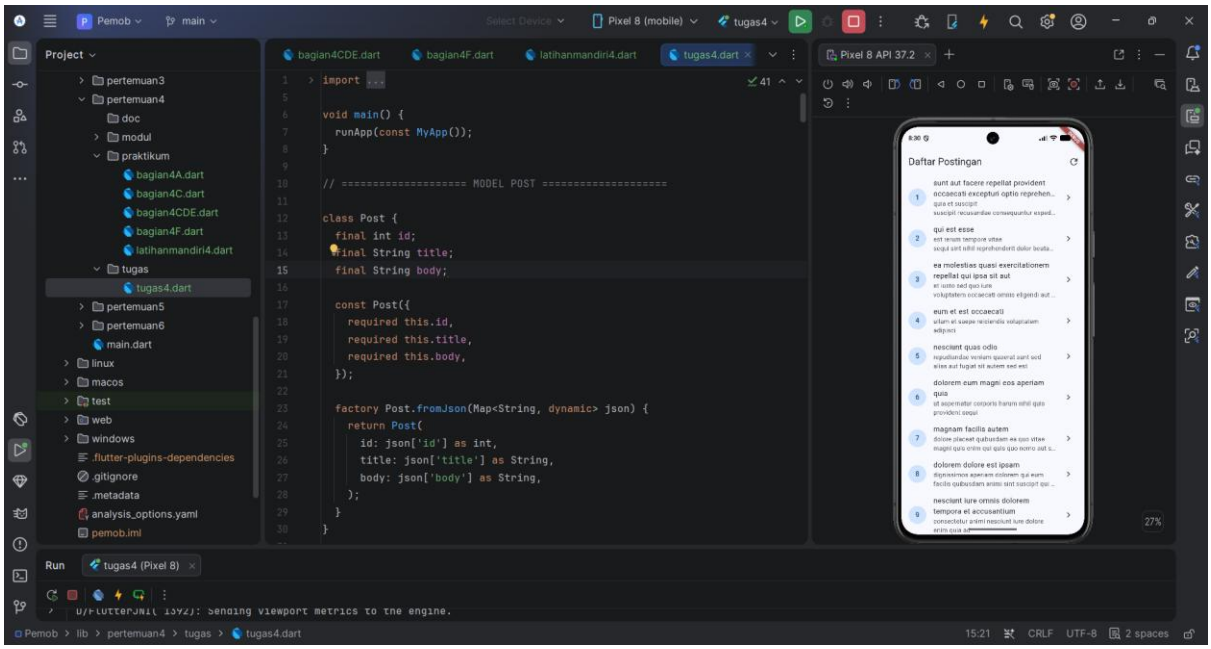
Latihan Mandiri





Pada bagian latihan mandiri ini, aplikasi dikembangkan lebih lanjut dengan menambahkan atribut tambahan seperti *username* serta data bersarang kota pada model dan halaman detail pengguna. Selain itu, antarmuka diperbarui dengan menampilkan jumlah total pengguna pada *AppBar*, serta menerapkan fitur *RefreshIndicator* dan penanganan kondisi daftar kosong untuk mengoptimalkan fungsionalitas dan pengalaman penggunaan aplikasi.

Tugas



Pada bagian tugas ini, aplikasi dikembangkan untuk menampilkan daftar postingan beserta potongan isinya dari endpoint /posts menggunakan FutureBuilder. Ketika salah satu postingan dipilih, aplikasi membuka halaman detail yang memuat isi lengkap serta daftar komentar dari endpoint terpisah (/posts/{id}/comments), lengkap dengan pemisahan kode model dan penanganan status pemuatan maupun galat pada kedua halaman tersebut.

Pertanyaan Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()? Mengapa?

   Jawab: Jika Future dibuat di dalam build(), proses pengambilan data akan berulang kali terpicu setiap kali widget mengalami *rebuild*, yang menyebabkan aplikasi melakukan permintaan data secara tidak perlu dan berpotensi memicu galat atau perulangan tak terbatas

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

   Jawab: snapshot.hasError mendeteksi kegagalan secara menyeluruh pada tingkat asinkron (seperti masalah koneksi atau waktu habis), sedangkan pemeriksaan response.statusCode memvalidasi respons spesifik dari server (seperti kode 404 atau 500); keduanya diperlukan agar aplikasi dapat menangani gangguan jaringan maupun kesalahan pada sisi peladen secara akurat.

3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

   Jawab: Data JSON sebaiknya diubah menjadi *class model* agar struktur data lebih terdefinisi dengan jelas, aman dari kesalahan pengetikan kunci, serta mempermudah

pengelolaan atribut menggunakan tipe data yang pasti dibandingkan langsung menggunakan *Map*.

4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

   Jawab: UI wajib menyediakan status *loading* dan *error* agar pengguna mengetahui bahwa proses sedang berjalan atau terjadi kendala jaringan, sehingga antarmuka tidak terlihat macet dan memberikan pengalaman penggunaan yang informatif serta interaktif melalui tombol coba lagi.

Kesimpulan

Kesimpulan praktikum modul ini adalah mahasiswa berhasil memahami dan menerapkan konsep pemrograman asinkron, penggunaan paket *http* untuk mengambil data dari *REST API*, serta pemetaan format *JSON* ke dalam bentuk *class model*. Selain itu, praktikum ini melatih keterampilan dalam mengelola antarmuka secara dinamis menggunakan FutureBuilder serta menangani berbagai kondisi status seperti pemuatan, galat, dan ketersediaan data secara efektif.